

An Analysis on Semantic Context to Assist in Secure Code Generation

Anonymous Author(s)

1 Overview

In recent years, IT engineers have increasingly adopted large language models (LLMs) to accelerate the coding process, whether it be a programming language such as Python or an Infrastructure-as-Code (IaC) language such as Terraform. By 2023, GitHub reported that GitHub Copilot contributed to 46% of source code developed by its users [2]. However, while LLMs generate code rapidly, LLM-generated code often contains critical security vulnerabilities. Gong et al. looked at some of the top models for code generation in 2025 across 121 Python code completion tasks and found that, on average, 76.2% of the generated code was vulnerable [3]. Anjum evaluated LLM-generated IaC across 21 studies (analyzing 600 Terraform files generated by five distinct models) and found that 68% contained HIGH or CRITICAL warnings when evaluated by static security scanners like Checkov [1]. Similarly, Vargas et al. demonstrated that while LLMs can produce working IaC, they often had little to no compliance with security checkers [7].

Prior work has improved the security of LLM-generated code, including IaC, mostly through fine-tuning the model or giving it additional context. The successful approaches share a pattern: they tend to supply the model with the security-relevant knowledge, rather than just examples of valid syntax. For example, Reyes et al. collected Terraform scripts from 1,466 open-source repositories, performed a vulnerability assessment to extract security context, and used these findings to fine-tune two open-source LLMs, resulting in substantial security gains [5]. To give another example, Shi & Zhang proposed a RAG (retrieval-augmented generation) framework that includes semantic information along with static program analysis, and improved security outcomes for 6 LLMs by 4.8% (using the SecurePass@1 metric) while retaining 98.7% of the models' original ability to generate working code [6].

We argue that, to improve an LLM's output, added context should go beyond just syntax and instead should also incorporate semantics. Syntax determines if code is well-formed; semantics determines what the code actually does when it runs. LLMs have become very good at generating syntactically valid code, but it's much easier for them to miss semantic constraints, resulting in bugs and insecure setups. Giving the LLM those constraints as part of the prompt gives the model info it cannot infer from syntax alone.

This brings us to Zodiac, a tool that mines semantic checks from online IaC repositories and validates them via deployment-based testing [4]. This tool by itself is not primarily LLM-based (except in a few cases where LLMs are used to fill in missing information), nor does it attempt to address problems relating to insecure Terraform code. However, its authors suggest that the checks "could serve as a RAG (retrieval-augmented generation) knowledge base to provide additional context to LLMs. By explicitly asking LLM models to conform with these checks, users could potentially improve the quality of their generated programs and accelerate development." [4].

For our project, we propose exploring how semantic context can affect the security of LLM-generated IaC, measuring how a Zodiac-informed LLM does security-wise compared to a normal LLM. For this process, we plan to obtain the required context by deploying and using Zodiac in a similar manner to the original authors, obtaining information for 52 popular Azure resources (including Virtual Machines, Virtual Networks, etc.). This process requires the use of Microsoft Azure. We plan to use the Azure for Students plan for this. This may limit the extent to which we can test things, especially if we run out of credits for anything we deploy.

Zodiac's checks target deployment correctness rather than security, so it is not immediately obvious that they will reduce the findings reported by security scanners. Our hypothesis is that they will, because many insecure deployments arise from the same misunderstanding of IaC that causes deployment failures. A result showing no improvement would also provide information, since it would point to the idea that correctness-facing context doesn't transfer to security.

To evaluate the impact, we will prompt both a baseline LLM and a Zodiac-informed LLM to generate Azure Terraform code across various data types. The generated code will first be checked by Zodiac to ensure semantic correctness and working execution. Next, we will evaluate the code using static security analysis tools such as Checkov and Trivy. We will measure whether the code passes, count total warnings, and track if there is any reduction in high-severity vulnerabilities. Since there is variability, we will run these tests multiple times and average the results.

Another avenue we are considering on top of this is to compare different LLMs for code generation (for example, Claude Fable against Gemini Pro). Since not every model is created equal, it's worth exploring how Zodiac's added context may affect different models, if at all. Furthermore, we are considering comparing the different versions of the same model (for example, the model version as of 6 months ago vs. 1 year ago). The specific LLMs we plan to use are subject to availability.

To summarize our proposal:

- We plan to evaluate how incorporating Zodiac's semantic checks as context impacts the overall security of LLM-generated Azure Terraform code.
- We hypothesize that correctness-facing semantic context reduces security findings.
- We also plan to measure how the effect of Zodiac input differs across LLMs.
- Security of generated code will be measured by using scanners like Checkov and Trivy.

References

- [1] Mateen Ali Anjum. 2026. Treat LLM-Generated Terraform as Untrusted: An Exploratory Security Study and a Practitioner's Verification Workflow. *IEEE Software* (2026), 1–7. doi:10.1109/MS.2026.3721790

- [2] Thomas Dohmke. 2023. The economic impact of the AI-powered developer lifecycle and lessons from GitHub Copilot. *GitHub Blog* (2023).
- [3] Jianian Gong, Nachuan Duan, Ziheng Tao, Zhaohui Gong, Yuan Yuan, and Minlie Huang. 2025. How Well Do Large Language Models Serve as End-to-End Secure Code Agents for Python?. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE '25)*. Association for Computing Machinery, New York, NY, USA, 1004–1013. doi:10.1145/3756681.3756984
- [4] Yiming Qiu, Patrick Tser Jern Kon, Ryan Beckett, and Ang Chen. 2024. Unearthing Semantic Checks for Cloud Infrastructure-as-Code Programs. In *Proceedings of the 30th ACM SIGOPS Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery. doi:10.1145/3694715.3695964
- [5] Raul Y. Reyes, Benjamin M. Ampel, and Hsinchun Chen. 2025. Large Language Models for Infrastructure as Code Vulnerability Remediation. In *Proceedings of the 20th Pre-ICIS Workshop on Information Security and Privacy (WISP 2025)*. AIS Electronic Library (AISeL). <https://aisel.aisnet.org/wisp2025/1>
- [6] Jiahao Shi and Tianyi Zhang. 2026. RESCUE: Retrieval Augmented Secure Code Generation. arXiv:2510.18204 [cs.CR] <https://arxiv.org/abs/2510.18204>
- [7] Francis Luis Santos Vargas, Rodrigo Brandão Mansilha, and Diego Kreutz. 2026. Security-First Evaluation of Text-to-Terraform: Benchmarking LLMs and SLMs for Secure IaC Generation. arXiv:2608.02672 [cs.CR] <https://arxiv.org/abs/2608.02672>